ARCHITECTURE HANDBOOK

RiskPulse — Architecture Handbook

Where every layer lives, and where your new code belongs.

Generated from ARCHITECTURE.md

Rendered 28 September 2026 from the editable Markdown source · RiskPulse/AI/ARCHITECTURE.md

Contents

1. The architecture in one paragraph . . . 3
2. The layers and their folders . . . 3
3. Namespaces and file conventions . . . 4
4. The wire contract . . . 4
5. Validation lives in four places . . . 4
6. Data layer rules . . . 5
7. Frontend rules . . . 5
8. Decision table — where does my code go? . . . 6
9. The one feature recipe: standard CRUD . . . 7
10. Auth, permissions, and routing . . . 9
11. Traps that already exist in the codebase . . . 9
12. Pre-flight checklist . . . 10

> **How to use this file.** RiskPulse is a classic **ASP.NET Core MVC + Service Layer** app. Before you write any code, find your concern in the decision table (§8) and put the code in the layer named there. Follow §9 for the shape of a new feature. If reality differs from this document, fix the document — do not invent a second pattern.

1. The architecture in one paragraph

A browser request hits a **Controller** (thin, one action per endpoint). The controller asks a **Service** to do the work. The service talks to **EF Core** and returns either an `ApiResponse<T>` (JSON for AJAX) or a **ViewModel** (for rendering a page). The **View** is a Razor page that renders HTML and calls the same endpoints again via `RiskPulse.*` JavaScript helpers. **Entities** mirror the database tables 1:1 and never leave the data layer. **DTOs** carry data between layers; **ViewModels** shape data for a specific page.



Not used here: repositories, CQRS/MediatR, DDD aggregates, AutoMapper, unit-of-work abstractions, or an API layer. Services query `AppDbContext` directly — that is intentional.

2. The layers and their folders

Layer	Folder	Owns	Never does
Presentation	`Views/{Controller}/`, `wwwroot/js/modules/riskpulse.js`, `wwwroot/css/site.css`	Markup, DataTables config, client validation, modals, toasts	Business rules, DB access, hand-rolled `$.ajax`
HTTP / Endpoint	`Controllers/`	Routing, `[Authorize]`, ModelState → JSON, wrapping in `ApiResponse`	Business rules, EF queries, view shaping
Application / Service	`Services/{Group}/`	Business rules, validation, transactions, orchestration, `ApiResponse`/ViewModel construction	Raw HTTP concerns, View markup
Domain / Entities	`Data/Entries/`	1:1 table mirrors + navigation properties	Behaviour, validation, DTOs
Contracts	`Models/Dto/`, `Models/ViewModel/`, `Models/Enum/`	Wire shapes (`*SaveDto`, `ApiResponse<T>`), page shapes (`*GridRowViewModel`), enums	Data access, logic
Infrastructure / Data	`Data/AppDbContext.cs`, `Data/Extensions/`, `Data/Seed.sql`	EF mapping, delete behaviours, indexes, schema, seed data	Business rules, HTTP

Service groups mirror the business area, not the technical type:

```
Services/Login/          AdAuthenticationService, DbAuthorizationService,
                         LoginOrchestratorService, PermissionCatalog, PermissionPageMapper
Services/Administration/ UsersService, RolesService, UnitsService
Services/Templates/      SaqTemplatesService, KriTemplatesService
Services/Schedule/       ScheduleService
Services/Assessment/     AssessmentService, SubmissionsService
Services/Dashboard/      DashboardService
Services/Utilities/      CodeGeneratorService
```

Dependency rule (the only one that matters): dependencies point inwards only — View → Controller → Service → Data. Never the reverse. A service may use another service (e.g. `ScheduleService` calls `AssessmentService`); an entity, DTO, or ViewModel may never reference a service instance.

3. Namespaces and file conventions

Kind	Folder	Namespace style
Controllers, Services, helpers	`Controllers/`, `Services/`	File-scoped: `namespace RiskPulse.Controllers;`
Entities, DTOs, ViewModels, Enums	`Data/Entries/`, `Models/*/`	Block: `namespace RiskPulse.Models.Dto { }`

Suffixes carry meaning. `*Service`, `*SaveDto`, `*GridRowViewModel`, `*IndexViewModel`, `*OptionViewModel`, `ApiResponse<T>`. If a new class has none of these suffixes, it probably does not belong.

Comments. Controllers and services use `// --- Section name ---` banners. Property-bag classes (entities, DTOs, ViewModels) get no comments. Views use `/* ====== SECTION n — ... */`.

4. The wire contract

Every JSON endpoint returns the same envelope (`Models/Dto/ApiResponse.cs`):

```
{ "success": true,  "message": "Unit updated successfully.", "data": { "id": 4 } }
{ "success": false, "message": "Unit code 'FIN' already exists.", "errors": null }
```

Rules:

- Produced by `ApiResponse.Ok(data, message?)` / `ApiResponse.Fail(message, errors?)`.
- JSON is **camelCase**; enums serialize as **strings** (`"status": "Draft"`), because `Program.cs` registers `JsonStringEnumConverter`.
- The browser only ever checks `success` — `RiskPulse.postJson` / `getJson` do that for you.
- `Id == 0` on a `*SaveDto` means **insert**; anything else means **update**.
- Delete always posts the shared `DeleteRequestDto { Id }`.

5. Validation lives in four places

Each layer owns a different class of rule. Never let a lower layer guess what a higher layer meant.

Layer	Owns	Example
View (JS)	UX only: highlight, focus, friendly message	`RiskPulse.validationError($('#unitCode'), 'Unit code is required.')`
DTO attributes	Shape: required, length, range, format	`[Required] [StringLength(20)] [Range(1, int.MaxValue)]`
Service	Business rules, cross-entity, state, uniqueness	`throw new InvalidOperationException("A group must have at least 2 units.")`
Database	Integrity backstop: FK, unique index, not-null	unique index on `(GroupId, UnitId)`

One rule for errors: a user-facing failure is a business rule, so it is an `InvalidOperationException` thrown in the service. Controllers must never catch it themselves — `ControllerHelpers` converts it to `ApiResponse.Fail`. Never validate by returning a "failed" result object.

6. Data layer rules

`Data/Entries/*.cs` mirror `riskpulse.*` tables 1:1 — no base classes, no interfaces, no behaviour, no DTOs inside an entity.

Adding an entity means all four of these in `Data/AppDbContext.cs`:

```
public DbSet<Widget> Widgets => Set<Widget>();          // 1. DbSet on AppDbContext

modelBuilder.Entity<Widget>(entity =>                   // 2. mapping inside OnModelCreating
{
    entity.HasKey(w => w.WidgetId);
    entity.Property(w => w.WidgetStatus)                 //    enums are varchar(32)
        .HasConversion<string>()
        .HasMaxLength(32);
    entity.HasIndex(w => w.WidgetCode).IsUnique();
    entity.HasOne(w => w.Unit)                           //    always state the delete behaviour
        .WithMany()
        .HasForeignKey(w => w.UnitId)
        .OnDelete(DeleteBehavior.Restrict);
});
```

- `HasDefaultSchema("riskpulse")` — the schema is set here **and** via `Search Path` in the connection string.
- Every enum property needs `HasConversion<string>()` + `HasMaxLength(32)`. No exceptions.
- Delete behaviour is explicit, always: `Cascade` for owned children, `Restrict` for reference data (Unit, Group, User, Workflow) and for any table another table depends on.
- `Migrations/` is git-ignored. Schema truth lives in the live DB plus `Data/Seed.sql`.
- Reusable query helpers go in `Data/Extensions/DbSetExtensions.cs`: `EnsureUniqueAsync` (throws `InvalidOperationException` on a duplicate) and `ToOptionListAsync` (projects to `List<OptionViewModel>`).

7. Frontend rules

`wwwroot/js/modules/riskpulse.js` (loaded by `_Layout`) is the only place shared browser behaviour lives. **Call it; never re-implement it.**

Need	Call
POST / GET JSON with envelope handling + flight lock	`RiskPulse.postJson(url, payload, handlers, $trigger)` / `getJson(...)`
Success / error toast	`RiskPulse.toastSuccess(msg)` / `toastError(msg)`
Field-level validation UX	`RiskPulse.validationError($el, msg)` / `clearFieldErrors($form)`
Read a form (checkboxes → bool, numerics coerced)	`RiskPulse.serializeForm($form)`
Fill a dropdown	`RiskPulse.populateSelect($sel, items, placeholderValue, placeholderLabel)`
Enhance a `<select class="rp-input">`	`RiskPulse.initSelect2(selector)` — never `$el.select2()` directly
Open / close a modal	`RiskPulse.showModal(id)` / `hideModal(formEl)` — never `$('#x').modal('show')`
Delete confirmation	`RiskPulse.confirmDelete(title, text, onConfirmed)`
DataTable with shared defaults	`RiskPulse.initGrid($table, cfg)`
Cell rendering	`RiskPulse.pill(label, kind)`, `statusPill(status)`, `statusKind(status)`, `escapeHtml(v)`

View rules:

- A page script is an IIFE `(function ($) { ... })(jQuery);` holding **only** validation rules, DataTables `columns`, and event wiring, split into `/* ====== SECTION n — ... ====== */` banners.
- `@section Styles` for page CSS, `@section Scripts` for the JS block.
- Grid buttons carry a stable class (`edit-unit-btn`, `delete-unit-btn`) and are wired by delegation on `'#unitTable tbody'`.
- DataTables `columns[].data` keys are camelCase and must match the `*GridRowViewModel` property names exactly.
- Server-rendered initial dropdown data is injected with `@Html.Raw(Json.Serialize(items))` — no AJAX for first paint.
- CSS is design-system only: `rp-btn-primary`, `rp-btn-edit`, `rp-btn-delete`, `rp-pill-success`, `rp-input`, `rp-card`, `rp-table`. All new classes are prefixed `rp-`. No new `site.css` outside the existing token/theme blocks.

8. Decision table — where does my code go?

My concern	Goes in
A new page	`Views/{Controller}/Index.cshtml` + `Views/Shared/_Layout.cshtml` nav link
A new AJAX endpoint	An action in the matching `Controllers/{Name}Controller.cs`
"Should this be allowed?" / uniqueness / state rules	The service
A table	An entity in `Data/Entries/` + `DbSet` + `OnModelCreating` block
A rule about *this request's shape* (length, required)	The `*SaveDto` attributes
A column of the grid	The `*GridRowViewModel`

My concern	Goes in
Initial data a page needs	A `*IndexViewModel` passed from the controller `Index` action
A dropdown's id/label pairs	`ToOptionListAsync` → `List<OptionViewModel>`
A generated key (`SAQ-yyyyMMdd-0001`)	`CodeGeneratorService`
A new page permission	`PermissionCatalog` + `Program.cs` policy + `PermissionPageMapper` + seed + sidebar
Reusable browser behaviour	`riskpulse.js`
A new approval step / status	A `WorkflowStep` row in the seed — **not** a C# enum
A new value like `UnitType`	`Models/Enum/` + `HasConversion<string>()`

9. The one feature recipe: standard CRUD

Copy this shape for any new admin entity. Example: a new `Widget` entity.

Step 1 — Data. `Data/Entries/Widget.cs` (plain properties, `WidgetId` PK), `DbSet<Widget>` in `AppDbContext`, one `modelBuilder.Entity<Widget>` block (§6).

Step 2 — Contracts.

- `Models/Dto/WidgetSaveDto.cs` with DataAnnotations and `Id == 0` meaning insert.
- `Models/ViewModel/WidgetGridRowViewModel.cs` with exactly the grid's columns.
- Reuse `DeleteRequestDto`, `SaveResultDto`, `OptionViewModel`.

Step 3 — Service in the matching `Services/{Group}/` folder:

```
public async Task<List<WidgetGridRowViewModel>> GetGridRowsAsync()
{
    return await _db.Widgets.AsNoTracking().OrderBy(w => w.WidgetId)
        .Select(w => new WidgetGridRowViewModel { WidgetId = w.WidgetId, WidgetCode = w.WidgetCode })
        .ToListAsync();
}

public async Task<SaveResultDto> SaveWidgetAsync(WidgetSaveDto model)
{
    var code = model.WidgetCode.Trim();
    await _db.Widgets.EnsureUniqueAsync(w => w.WidgetCode.ToLower() == code.ToLower()
                                          && w.WidgetId != model.WidgetId, "Widget code", code);

    if (model.WidgetId == 0)                                        // insert
    {
        var widget = new Widget { WidgetCode = code };
        _db.Widgets.Add(widget);
        await _db.SaveChangesAsync();
        return new SaveResultDto { Id = widget.WidgetId };
    }

    var existing = await _db.Widgets.FindAsync(model.WidgetId)
        ?? throw new InvalidOperationException($"Widget with Id {model.WidgetId} was not found.");
    existing.WidgetCode = code;
    await _db.SaveChangesAsync();
    return new SaveResultDto { Id = existing.WidgetId };
}

public async Task DeleteWidgetAsync(int id)
{
    var widget = await _db.Widgets.FindAsync(id)
        ?? throw new InvalidOperationException($"Widget with Id {id} was not found.");
    if (await _db.Something.AnyAsync(s => s.WidgetId == id))        // in-use guard
        throw new InvalidOperationException("Cannot delete a widget that is in use.");
```

```
    _db.Widgets.Remove(widget);
    await _db.SaveChangesAsync();
}
```

Step 4 — Controller. Thin, one action per verb, no rules:

```
[Authorize(Policy = $"Permission:{PermissionCatalog.Widgets}")]
public class WidgetsController : Controller
{
    private readonly WidgetsService _widgetsService;
    public WidgetsController(WidgetsService widgetsService) => _widgetsService = widgetsService;

    [HttpGet] public async Task<IActionResult> Index()
        => View(new WidgetsIndexViewModel { Widgets = await _widgetsService.GetAllAsync() });

    [HttpGet] public async Task<IActionResult> Grid()
        => Json(ApiResponse.Ok(await _widgetsService.GetGridRowsAsync()));

    [HttpPost] public async Task<IActionResult> SaveWidget([FromBody] WidgetSaveDto model)
        => await ControllerHelpers.TrySave(model, ModelState, m => m.WidgetId,
            m => _widgetsService.SaveWidgetAsync(m), "Widget");

    [HttpPost] public async Task<IActionResult> DeleteWidget([FromBody] DeleteRequestDto request)
        => await ControllerHelpers.TryDelete(request, ModelState,
            id => _widgetsService.DeleteWidgetAsync(id), "Widget");
}
```

`ControllerHelpers.TrySave` / `TryDelete` already null-guard, validate ModelState, and turn `InvalidOperationException` into `ApiResponse.Fail`. For a non-CRUD action, use `ControllerHelpers.TryExecute(async () => ..., "generic message")`. **Never write a try/catch or a validation block in a controller.**

Step 5 — View `Views/Widgets/Index.cshtml`: a `rp-card` + `<table id="widgetTable">`, `@section Styles` (DataTables CSS), `@section Scripts` (DataTables JS + IIFE):

```
(function ($) {
    /* ====== SECTION 1 — ENDPOINTS ====== */
    var gridUrl = '@Url.Action("Grid", "Widgets")';

    /* ====== SECTION 2 — GRID ====== */
    var table = RiskPulse.initGrid($('#widgetTable'), {
        ajax: { url: gridUrl },
        columns: [
            { data: 'widgetCode' },
            { data: 'widgetId', orderable: false, searchable: false, className: 'text-end',
              render: function () {
                  return '<button class="rp-btn rp-btn-edit edit-widget-btn"><i class="fas fa-pen"></i></button>' +
                         '<button class="rp-btn rp-btn-delete delete-widget-btn"><i class="fas fa-trash"></i></button>';
              } }
        ]
    });

    /* ====== SECTION 3 — EVENTS ====== */
    $('#widgetTable tbody').on('click', '.delete-widget-btn', function () {
        var row = table.row($(this).closest('tr')).data();
        RiskPulse.confirmDelete('Delete widget?', row.widgetCode, function () {
            RiskPulse.postJson(deleteUrl, { id: row.widgetId }, { success: function (r) {
                table.ajax.reload();
                RiskPulse.toastSuccess(r.message);
            } });
        });
    });
})(jQuery);
```

Step 6 — Wire it up: `builder.Services.AddScoped<WidgetsService>();` in `Program.cs`, plus the `AddPolicy($"Permission:{PermissionCatalog.Widgets}", ...)` line and a sidebar link.

Other feature shapes (same layers, different service logic):

- **Workflow transition** (submit / approve / authorize): the service loads the current `WorkflowStep`, checks the user's role, advances the step, and saves — statuses are `WorkflowStep.StepCode` rows, never C# enums.
- **Wizard step** (Schedule): each step posts to its own endpoint, then advances; the final step activates the schedule, which creates the assessment.
- **Page with no grid** (Dashboard): controller → service → `*IndexViewModel` → server-rendered Razor. No JSON needed.

10. Auth, permissions, and routing

- Cookie auth, 20-minute sliding expiry, `LoginPath=/Login/Index`. Claims: `Name`, `NameIdentifier` (int user Id), `Role`, `DefaultPage`, `Unit`, and one `Permission` claim per role permission.
- Every controller action is gated: `[Authorize(Policy = $"Permission:{PermissionCatalog.X}")]`.
- `PermissionCatalog` is the single source of truth for permission strings and must match the `Permissions.PermissionDesc` seed rows and the claims built in `LoginOrchestratorService` exactly.
- `PermissionPageMapper.GetRouteForPermission(desc)` maps a permission to its landing page (default: Dashboard). Renaming a page-backed permission means touching `PermissionCatalog`, the `Program.cs` policy, the mapper, the seed, the controller, the view folder, and the `_Layout.cshtml` nav link. A data-only permission needs none of that.
- `AdAuthenticationService` is a **dev stub** that accepts any password.

11. Traps that already exist in the codebase

Do not "fix" these silently; each is real, and two are live bugs.

1. **Workflow step codes are duplicated string literals** in several services and in `riskpulse.js`. `DashboardService.UnitAuthorizedStepCode = "Authorized"` does **not** match the seeded final step `FinalApproved`, so the dashboard's authorized counts read zero. A shared step-code catalog would fix it.
2. `AssessmentStatus` **is dead** — written once at creation, never read. Don't add logic on it.
3. **Ownership checks are hand-rolled** per service (`UnitId`, then role, then step). Reuse the pattern; don't skip it because a screen "looks" single-unit.
4. `Migrations/` **is git-ignored**, so a fresh database cannot be rebuilt from the repo.
5. **Grids use client-side processing** on every page — fine at this data volume, but a new grid with thousands of rows needs server-side paging.
6. `SubmissionsService` is large (three responsibilities). Prefer a new focused service over growing it further.

12. Pre-flight checklist

Before you call a feature done:

- [ ] Entity + `DbSet` + `OnModelCreating` block, enum conversions, explicit delete behaviour.
- [ ] `*SaveDto` carries the shape rules; the service carries the business rules.
- [ ] `*GridRowViewModel` keys match the DataTables `data:` names exactly.
- [ ] Service registered with `AddScoped<T>()` in `Program.cs`.
- [ ] Every action has `[Authorize(Policy = "Permission:...")]`; permission exists in `PermissionCatalog`, `Program.cs`, the seed, and (if page-backed) the mapper + nav.
- [ ] Controller has no `try/catch`, no rule, no EF query.
- [ ] View uses `RiskPulse.*` only — no `$.ajax`, no `$('#x').modal('show')`, no direct `.select2()`, no new un-prefixed CSS class.
- [ ] Errors reach the user as a toast from a service-thrown `InvalidOperationException`.
- [ ] `dotnet build` passes (build is the only automated verification in this repo).